

Vectorization Process Explained

A code-level guide to **src/vectorization**: how the project turns a 7-class raster segmentation preview into clean CAD-like primitives and final SVG output.

The vectorizer is not another neural network. It is a deterministic geometry pipeline that starts from a color/class mask, cleans each class, extracts walls/openings/doors/floor, applies architectural constraints, resolves scale when evidence is good enough, and writes an SVG plus debug artifacts.

1. Source Map

File	Role
run_mask_to_vector.py	Main orchestrator. Loads config, finds prediction images, runs decode -> cleanup -> wall/opening/door/floor extraction -> geometry rules -> scale -> SVG/debug/metrics. Main flow starts at process_single(), line 150.
decode_prediction.py, masks.py	Decode RGB prediction previews or class-id masks into six binary evidence masks: floor, wall, window, door_arc, door_leaf, door_origin.
cleanup.py	Small connected-component removal and light morphology. This suppresses speckle while preserving thin origin/leaf evidence.
wall_extraction.py	Builds the outer wall loop first, then inner walls from skeleton/Hough line evidence, then snaps dangling endpoints to nearby wall lines.
window_extraction.py	Finds window connected components, hosts them on nearest wall centerlines, and measures their width by projection along the wall.
door_extraction.py	Finds door_origin thresholds, hosts them on walls, chooses hinge and swing side from leaf/arc evidence density, then constructs symbolic leaf and arc primitives.
geometry_rules.py	Applies architectural cleanup: snap wall angles to 45-degree increments, re-project openings to snapped walls, and split walls at windows/doors.
primitives/	CAD-like primitive classes. Each stores geometry, confidence, evidence metadata, bounds, scale info, and to_svg() rendering.
wall_geometry.py	Shapely helpers that buffer centerlines into filled wall/window polygons and merge connected chains before buffering.
export_svg.py	Writes final SVG in group order floor -> wall -> window -> door; debug evidence is excluded from final SVG.

2. Major Frameworks And Roles

Framework	Role in vectorization
NumPy	Fast array representation for RGB images, class maps, masks, pixel coordinates, and projection math.
Pillow	Loads prediction images as RGB arrays and writes debug_overlay.png.
OpenCV (cv2)	Morphological closing, connected components, contour detection/simplification, distance transforms, and probabilistic Hough line detection.
scikit-image skeletonize	Turns wall masks into one-pixel centerline evidence before Hough line extraction.
Shapely	Merges connected line chains, buffers wall/window centerlines into filled polygons, unions those polygons, and converts them to SVG path geometry.
PyYAML	Loads configs/vectorization_v008.yaml, which controls thresholds, modules, output paths, and feature toggles.
ReportLab	Used only for this explanatory PDF, not by the vectorization source itself.

3. End-To-End Flow

1. **Find inputs:** `run()` reads `configs/vectorization_v008.yaml`, then `find_prediction_images()` looks under `runs/segformer_b0_run3/previews/epoch_030` for filenames containing prediction.
2. **Decode mask:** `decode_color_mask()` maps RGB pixels to class ids 0..6 using the duplicated `run3` palette and a tolerance of 20. More than 5% unmatched pixels raises `IncompatibleMaskError`.
3. **Split masks:** `split_class_masks()` creates binary masks for floor, wall, window, door_arc, door_leaf, and door_origin.
4. **Clean evidence:** `cleanup` removes small connected components; walls and door arcs get light closing, while door_origin and door_leaf stay thin to preserve measurable line length.
5. **Build walls first:** `extract_walls()` creates the outer loop from wall plus opening evidence, then extracts inner walls from the remaining wall mask.
6. **Host openings:** windows and door_origin components attach to the nearest wall if within `max_host_wall_dist_px`, then their pixels are projected onto that wall to get clean endpoints and width.
7. **Generate doors:** for each hosted door origin, nearby door_leaf/door_arc evidence picks the hinge endpoint and swing side. The leaf and arc are procedural, not raw contour traces.
8. **Apply geometry rules:** wall segments snap to cardinal or diagonal angles, hosted openings are re-projected, door geometry is re-anchored, and walls are split at opening spans.
9. **Resolve scale:** door widths and wall thickness samples vote against known modules. If explicit scale or consistent evidence exists, primitives receive millimeter annotations; otherwise geometry remains pixel-space.
10. **Build floor and export:** the floor is the outer wall loop polygon. `build_svg()` writes floor, unified wall polygon, windows, and grouped doors; `debug_overlay.png` and `metrics.json` are side outputs.

4. Important Decisions Encoded In The Code

The floor mask is intentionally not trusted for the building envelope

`wall_extraction.py` states that the outer loop is built from wall plus opening evidence, never floor evidence. The reason is practical: comments reference task08 and say the floor class is the CNN's least accurate class. So the floor in the final vector is derived from the outer wall polygon, not from the predicted floor mask.

Outer walls are solved before inner walls

The pipeline extracts a closed exterior envelope first, wraps it as `OuterWallLoopPrimitive`, then erases entire wall connected-components touched by that loop before running inner-wall detection. This avoids rediscovering fragments of the exterior wall as duplicate inner walls.

Openings must be wall-hosted

Windows and door origins are not accepted just because a colored blob exists. They must find a nearby host wall, then their own pixels are projected onto that wall centerline. Too-narrow or unhosted blobs become unresolved debug markers and are excluded from the final SVG.

Orthogonal interpretation wins unless diagonal evidence is explicit

`snap_walls_to_45()` snaps to horizontal/vertical first when within 20 degrees. A 45-degree wall is used only when evidence is within 10 degrees of an exact diagonal. Ambiguous cases fall back to the nearest cardinal direction because floor plans are usually orthogonal and noisy pixels should not invent diagonals.

Doors are reconstructed symbolically

The door_origin line is measured from mask evidence, but the leaf and arc are generated from geometry rules. The hinge is the origin endpoint with more nearby door_leaf/door_arc evidence; swing side is chosen by probing both perpendicular directions from the hinge.

5. Important Parameters

Parameter	Value / behavior
decode tolerance	decode_color_mask(rgb, tolerance=20). Pixels farther than 20 RGB units from the palette are unmatched; >5% unmatched rejects the image as incompatible.
cleanup min areas	wall 20, floor 100, window 8, door_arc 4, door_leaf 4, door_origin 4. These remove tiny segmentation noise.
close_wall_gap_px	3 px rectangular morphology close, used before wall component filtering.
merge_distance_px	6 px. Used as Hough maxLineGap and collinear merge distance for inner wall segments.
min_wall_length_px	10 px. Inner wall Hough lines shorter than this are ignored.
connect_gap_px	20 px. Dangling inner-wall endpoints can snap to nearby wall lines within this distance.
max_host_wall_dist_px	40 px. Maximum distance for a window or door_origin component to attach to a wall.
min_hosted_width_px	10 px. Hosted window/door spans narrower than this are treated as unresolved/noise.
ortho_snap_degrees	20 degrees. Cardinal directions have priority.
diagonal_snap_degrees	10 degrees. Diagonal walls require close evidence.
door modules	600, 800, 900 mm. Used for scale voting and snapping door widths.
wall modules	100, 200 mm. Used for scale cross-checks and wall thickness snapping.
window modules	600, 900, 1200, 1500 mm. Used only when scale is trustworthy enough.
metric confidence	min_scale_confidence_for_metric = 0.70. Below this, primitives keep pixel geometry and mm fields remain unset.
module tolerance	15% relative error. Measurements within this tolerance vote for or snap to a known module.
cross-check tolerance	25% relative difference. Door-derived and wall-derived px_to_mm estimates must agree this closely to be blended.

6. Raster-To-Vector Logic By Component

Input and class decoding

The active scheme is a strict 7-class run3 mask: background=0, floor=1, wall=2, window=3, door_arc=4, door_leaf=5, door_origin=6. The palette is duplicated inside decode_prediction.py to avoid importing heavier mask-generation dependencies just to decode preview images.

Wall extraction

Wall processing begins by estimating thickness with a distance transform over the skeletonized wall mask. The outer loop then uses wall and opening evidence, applies closing/dilation to bridge gaps, finds the largest external contour, simplifies it with approxPolyDP, and rectilinearizes each contour edge to horizontal or vertical. Each edge becomes an outer WallPrimitive.

Inner walls are extracted only after the exterior wall connected-components are erased. The remaining mask is skeletonized, HoughLinesP proposes centerlines, nearby collinear segments merge, short segments are dropped, and dangling endpoints snap to the nearby wall network.

Window extraction

Each window connected component is measured by its actual mask pixels, not by its bounding box. nearest_wall() chooses the host; project_pixels_onto_wall() projects all component pixels onto the host centerline. The min and max projection positions become the clean span. Window thickness is host wall thickness / 2, reflecting the code comment that a 100 mm window replaces a 200 mm wall.

Door extraction

Door_origin components go through the same host/project process as windows. Then the combined door_leaf + door_arc evidence is queried around each origin endpoint. The endpoint with greater evidence density becomes the hinge. The code probes six points along both perpendicular directions from the hinge; whichever side has more evidence becomes the swing side.

DoorLeafPrimitive and DoorArcPrimitive are then generated from hinge, far threshold point, width, wall orientation, and swing side. This is why the output is cleaner than the raw segmentation: the door symbol is reconstructed from architectural geometry instead of tracing noisy pixels.

Geometry cleanup and wall splitting

After windows and doors are hosted, walls snap to clean angles. Because snapping can move wall endpoints slightly, openings are re-projected onto their snapped host walls and door hinge/far points are re-hosted to the new origin endpoints. Finally, split_walls_at_openings() removes the opening intervals from each host wall and keeps the solid wall intervals on either side.

Scale resolution

Scale is explicit if config supplies explicit_px_to_mm. Otherwise, resolve_scale() votes for a px_to_mm factor by checking door_origin lengths against common door modules and wall thickness samples against wall modules. If both sources agree within 25%, it blends them 70% door, 30% wall. If they conflict, it refuses to guess and leaves unit=px.

SVG export

The wall group is not a pile of stroked lines. export_svg.py passes all wall centerlines to wall_geometry.segments_to_polygon(), which merges connected chains and buffers them with flat caps and mitred joins. The result is one or more filled black wall polygons. Floor is a white polygon behind everything. Windows are blue filled polygons. Doors are grouped origin/leaf/arc symbols.

7. What The Debug Outputs Mean

Artifact	Meaning
*_vector.svg	Final clean vector output. Contains only floor, wall, window, and door groups.
debug_overlay.png	Raster diagnostic overlay showing wall centerlines, outer loop, hosted windows/doors, unresolved evidence, and scale annotation.
metrics.json	Counts of outer/inner/final walls, resolved/unresolved windows and doors, floor presence, outer-loop closure, and scale status/source/confidence.

8. Key Code Anchors

Concern	Where to look
Pipeline order	src/vectorization/run_mask_to_vector.py:150 process_single(); run() starts at line 320.
Palette decoding	src/vectorization/decode_prediction.py:11 palette, line 31 decode_color_mask().
Binary mask split	src/vectorization/masks.py:18 split_class_masks().
Cleanup thresholds	src/vectorization/cleanup.py:18 and configs/vectorization_v008.yaml:47.
Outer wall loop	src/vectorization/wall_extraction.py:183 extract_outer_wall_loop().
Inner wall detection	src/vectorization/wall_extraction.py:296 extract_inner_walls().
Full wall extraction	src/vectorization/wall_extraction.py:348 extract_walls().
Wall angle snapping	src/vectorization/geometry_rules.py:17 snap_walls_to_45().
Opening projection	src/vectorization/geometry_rules.py:97 project_pixels_onto_wall().
Wall splitting	src/vectorization/geometry_rules.py:152 split_walls_at_openings().
Windows	src/vectorization/window_extraction.py:20 extract_windows().

Concern	Where to look
Doors	src/vectorization/door_extraction.py:88 extract_doors().
Scale	src/vectorization/primitives/scale.py:14 modules, line 84 resolve_scale().
Filled wall polygons	src/vectorization/wall_geometry.py:63 segments_to_polygon().
SVG output	src/vectorization/export_svg.py:47 build_svg().

9. Mental Model

Think of the vectorizer as a strict interpreter between a segmentation model and a drawing system. The model supplies colored evidence. The vectorizer asks architectural questions: What is the exterior envelope? Which wall hosts this opening? Is this length plausible? Is this angle likely cardinal? Does the door evidence identify a hinge? Only after those answers are stable does it draw clean primitives.

The main source of output clarity is constraint. The code deliberately avoids using weak evidence where stronger structural evidence exists, avoids generic opening classification because the 7-class model already separates classes, avoids metric claims when scale evidence conflicts, and avoids raw contour tracing for symbols such as doors.